RETO TÉCNICO

📦 Control de versiones

- El código debe ser almacenado en un repositorio **Git** creado por el candidato.
- El proyecto debe incluir un archivo **.gitignore** adecuado para proyectos de automatización (Java con Maven o Gradle), evitando versionar archivos generados.

📜 Consideraciones de implementación

- Se debe incluir una **historia de usuario**, redactada completamente en un solo idioma (español o inglés).
- La automatización debe ser implementada utilizando:
  - **Cucumber (BDD)** para la definición de escenarios
  - Patrón de diseño:
    - **Screenplay**
- Para la automatización de APIs se debe implementar:
  - **Karate DSL** como framework de automatización de servicios REST
  - Definición de escenarios en archivos .feature utilizando sintaxis Gherkin
  - Uso de variables dinámicas para el manejo de datos entre servicios
  - Implementación de flujo encadenado entre servicios (orquestación)
  - Validación de respuestas mediante assertions (match), incluyendo:
    - Código de respuesta HTTP
    - Estructura del response
    - Contenido de la respuesta

📐 Estructura y diseño

- El proyecto debe contar con una **estructura organizada y mantenible**, aplicando separación de responsabilidades.
- Se evaluará la correcta implementación del modelo de automatización.

🔍 Estrategia de localización de elementos

Se debe aplicar una estrategia adecuada para la identificación de elementos, priorizando:

1. ID
2. Name
3. Class
4. XPath relativos
5. CSS Selectors

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

🎯 Objetivo

Automatizar flujos funcionales de una aplicación web y servicios asociados, implementando buenas prácticas de automatización, diseño de framework y ejecución automatizada.

🌐 WEB (OBLIGATORIO)

Objetivo
Automatizar un flujo funcional end-to-end sobre la aplicación **Demo Web Shop Tricentis**, que incluya el proceso de registro de usuario y compra de productos, aplicando buenas prácticas de automatización, diseño de framework y validación funcional.

Aplicación bajo prueba

El robot deberá ingresar a la siguiente aplicación:

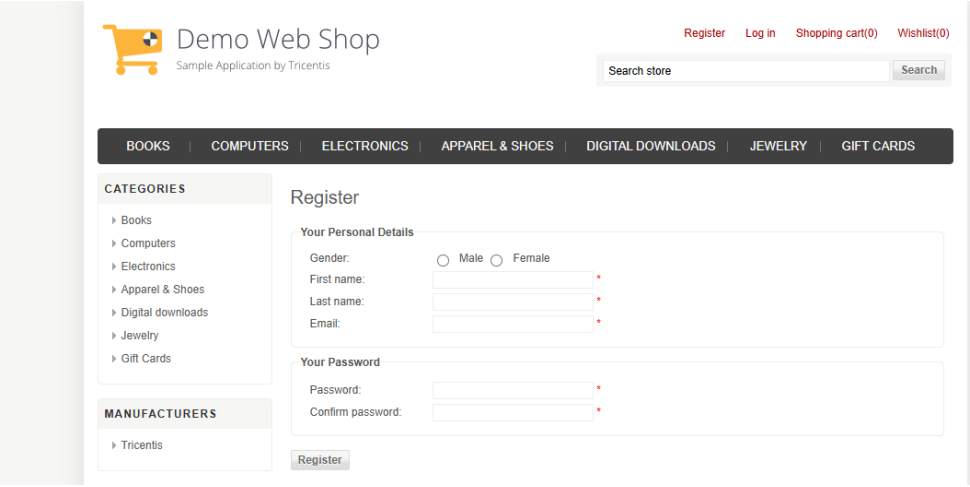
https://demowebshop.tricentis.com/

🧱 Flujo a automatizar

La automatización debe cubrir un flujo completo dividido en bloques funcionales:

BLOQUE 1: Registro de usuario

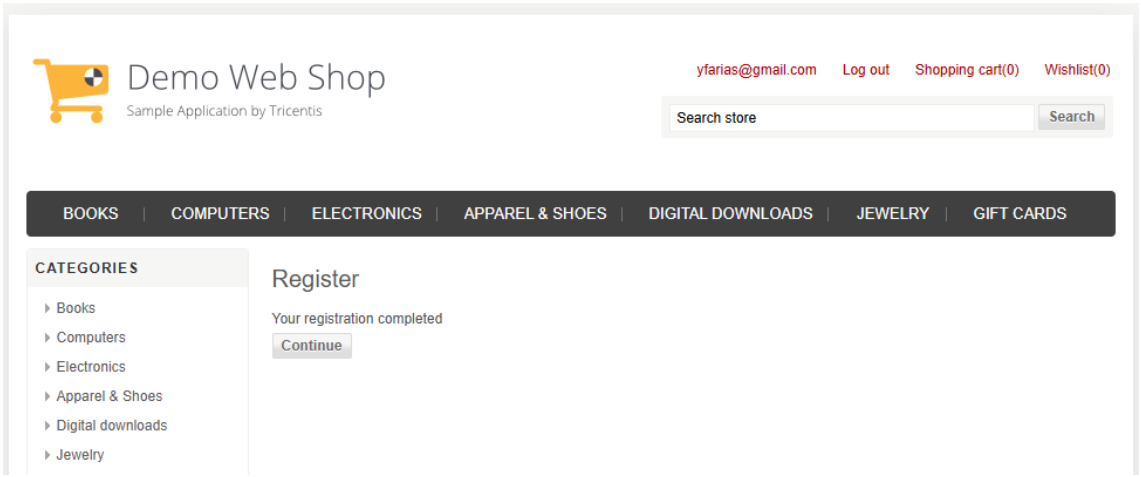
1. Dar clic en **Register** e ingresar datos correctos para registrase en la página.



CHOUCAIR® OK TESTED

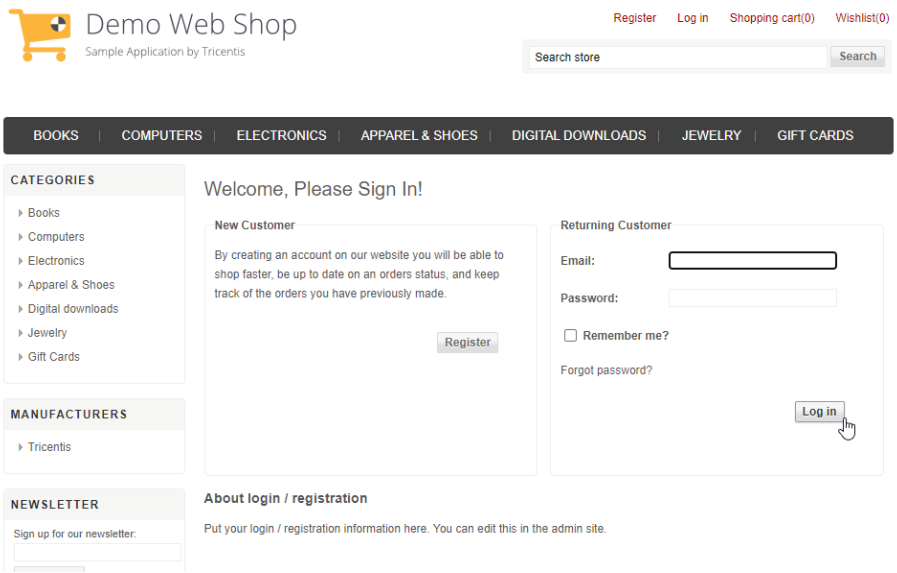
Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

2. Luego realizar la verificación del registro Exitoso



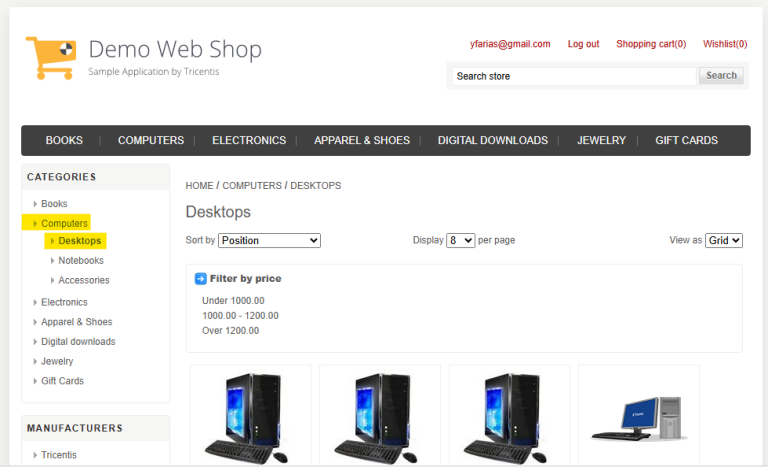
BLOQUE 2: Proceso de compra

3. Iniciar sesión en la Pagina Demo Web Shop Tricentis

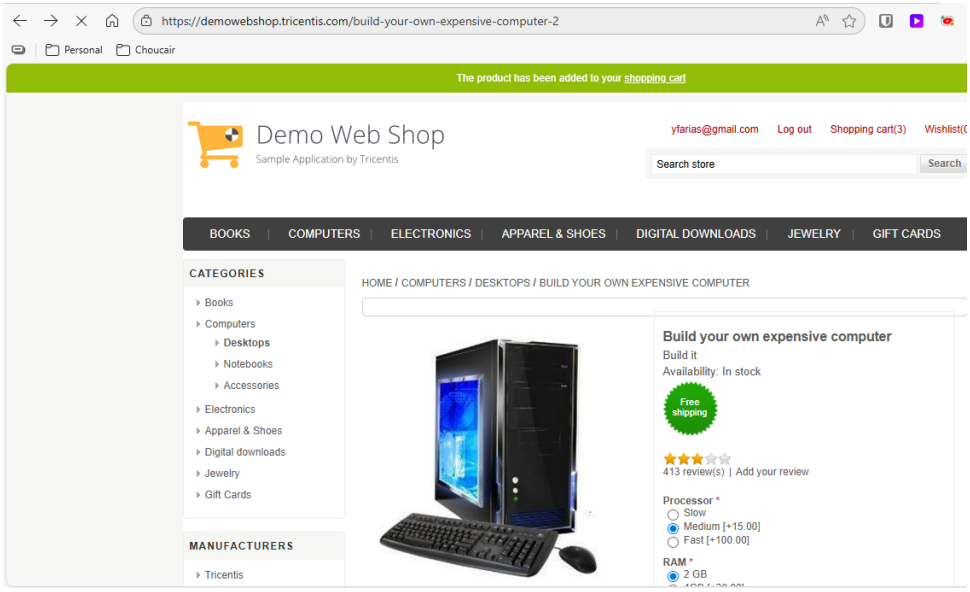


CHOUCAIR® OK TESTED

4. Dar clic en una Categoría>subcategoría (en caso de tenerla)

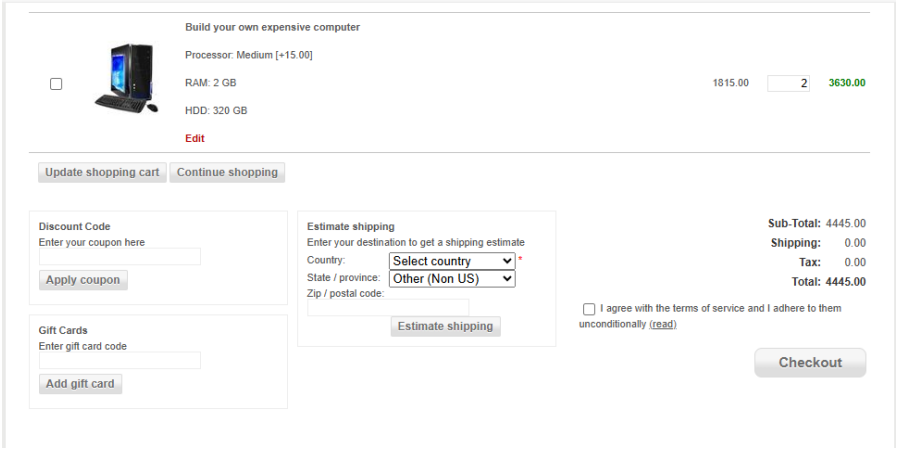


5. Agrega un producto al carrito

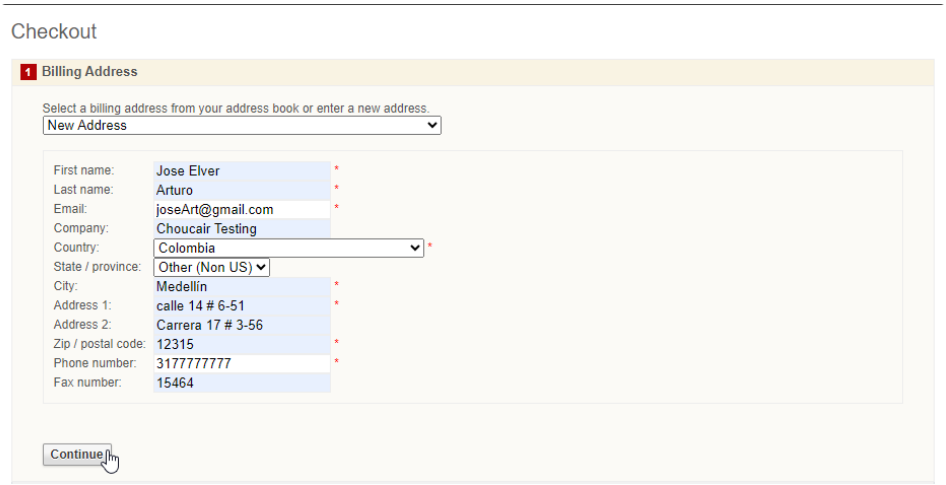


Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

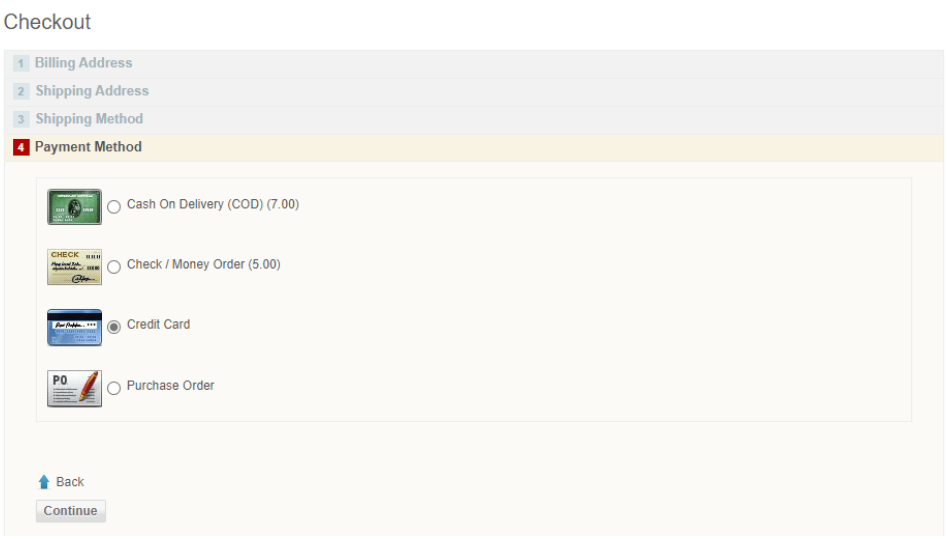
6. Ingresa al carrito y luego da clic en “Checkout”



7. Ingresa los datos solicitados para la entrega y los demás datos deje los que cargan por defecto

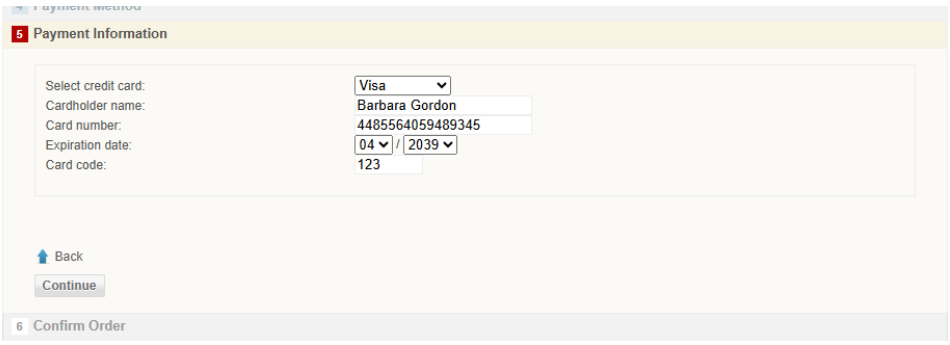


8. Selecciona tarjeta de crédito como método de pago



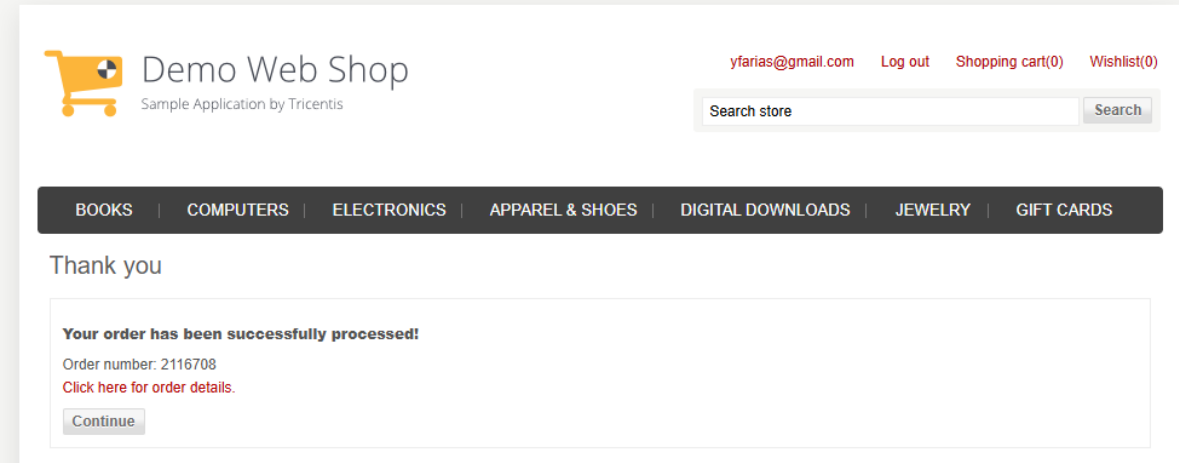
9. Completa la información de pago con los siguientes datos

Select credit card: Visa
Cardholder name: Barbara Gordon
Card number: 4485564059489345
Expiration date: 04/2039
Card Code: 123



Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

10. Confirme la orden y valide el mensaje "**Your order has been successfully processed!**"



🧪 Validaciones requeridas

Se deben implementar validaciones que aseguren:

1. Registro exitoso del usuario
2. Finalización exitosa de la compra

   Mensaje final:

   - "Your order has been successfully processed!"

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

APIS (OBLIGATORIO)

Objetivo
Automatizar un flujo completo sobre la API REST de **Tosca Cloud CoffeeShop**, validando operaciones CRUD y consistencia de datos mediante Karate DSL.

API base
https://webservice.toscacloud.com/api/docs/v1/

Flujo a automatizar (OBLIGATORIO)

BLOQUE 1: Creación

1. Crear un nuevo Coffee usando:

POST /api/v1/Coffees
Validar:

- Status code 200
- Estructura del response
- Persistencia del ID generado

BLOQUE 2: Consulta

2. Consultar el recurso creado:

GET /api/v1/Coffees/{id}
Validar:

- El objeto corresponde al creado
- Campos correctos

BLOQUE 3: Actualización

3. Actualizar el Coffee:

PUT /api/v1/Coffees
Validar:

- Cambios reflejados correctamente

BLOQUE 4: Eliminación

4. Eliminar el Coffee:

DELETE /api/v1/Coffees?id={id}
Validar:

- Status correcto

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

BLOQUE 5: Validación final

5. Intentar consultar el recurso eliminado

Validar:

- Recurso no existe o respuesta esperada

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

CI/CD

Objetivo del componente CI/CD
Implementar un pipeline en **Jenkins** o **Azure DevOps** que permita ejecutar de forma automatizada el proyecto del reto y evidencie la integración básica del framework de automatización dentro de un flujo de entrega continua.

Qué debe demostrar el pipeline
El pipeline debe evidenciar, como mínimo, que el candidato es capaz de:

- Obtener el código desde un repositorio Git
- Preparar el entorno de ejecución
- Compilar o preparar el proyecto
- Ejecutar las pruebas automatizadas
- Publicar o dejar evidencia de los resultados
- Dejar una configuración reutilizable y entendible

Implementación mínima esperada del pipeline
Etapas mínimas
Estas deberían ser obligatorias, tanto para Azure como para Jenkins:

1. **Checkout**
   Descarga del código fuente desde el repositorio.

2. **Preparación del entorno**
   Instalación o uso del runtime necesario:
   - Java
   - Dependencias del proyecto
   - Navegador o driver, si aplica para Web
   - Variables de entorno o parámetros básicos

3. **Build / Resolución de dependencias**
   Ejecución del comando de construcción según la herramienta:
   - Maven
   - Gradle

4. **Ejecución de pruebas**
   Lanzamiento del reto de automatización:
   - Web
   - API
   - o ambos, si aplica

5. **Publicación de resultados**
   Generación de evidencia visible de la ejecución:
   - resultado de pruebas
   - logs

CHOUCAIR® OK TESTED

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL

- reportes
- screenshots si aplica

Evidencias mínimas que debe entregar
Aquí está el punto clave: el pipeline no debe existir solo "por archivo", sino como evidencia de ejecución.

El candidato debe entregar al menos una de estas evidencias:
- Captura del pipeline ejecutado exitosamente
- Enlace al job o pipeline
- Archivo de configuración:
  - Jenkinsfile
  - azure-pipelines.yml
- Resultado visible de ejecución
- Publicación de reportes o artefactos

Qué debería ejecutar el pipeline según el tipo de reto
Para reto Web
Debe evidenciar:
- ejecución automatizada del flujo
- generación de reporte
- publicación de evidencias

Ejemplo de salida esperada:
- pruebas ejecutadas
- casos pasados/fallidos
- reporte Serenity, Cucumber o similar
- screenshots si el framework los genera

Para reto API
Debe evidenciar:
- ejecución de los feature de Karate
- resultado de escenarios
- reporte de ejecución

Ejemplo de salida esperada:
- Karate Reports
- logs de requests/responses
- resultado final del pipeline

Choucair Cárdenas Testing. Todos los derechos reservados - 2020. Choucair©
CONFIDENCIAL